

994. 腐烂的橘子

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 广度优先搜索, 数组, 矩阵
- **Companies:** (Not Available)

在给定的 $m \times n$ 网格 `grid` 中，每个单元格可以有以下三个值之一：

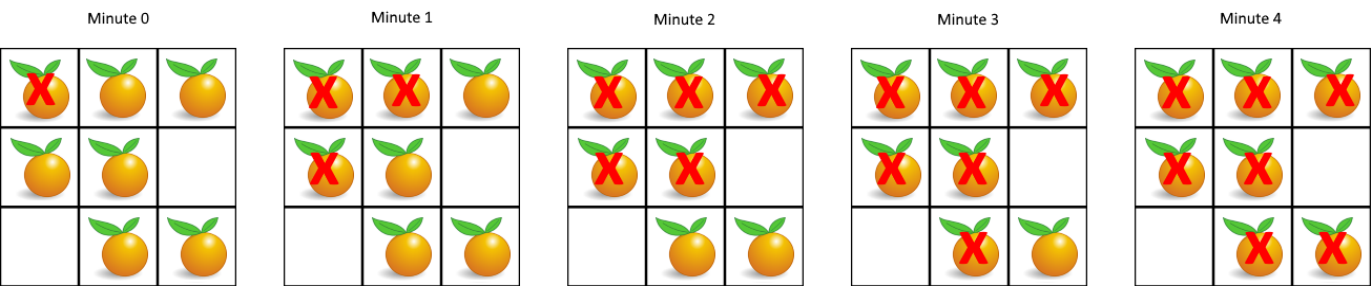
- 0 表示该单元格为空；
- 1 表示该单元格有一个新鲜橘子；
- 2 表示该单元格有一个腐烂橘子。

每分钟，腐烂的橘子 **周围** 的新鲜橘子（上、下、左、右四个方向）会腐烂。

返回 **直到** 单元格中没有新鲜橘子为止**所必须经过的最小分钟数**。如果不可能，返回 -1。

示例 1:

```
1 输入: grid = [[2,1,1],[1,1,0],[0,1,1]]
2 输出: 4
```



示例 2:

```
1 输入: grid = [[2,1,1],[0,1,1],[1,0,1]]
2 输出: -1
3 解释: 左下角的橘子（第 2 行，第 0 列）永远不会腐烂，因为腐烂只会发生在 4 个正向上。
```

示例 3:

```
1 输入: grid = [[0,2]]
2 输出: 0
3 解释: 因为 0 分钟时已经没有新鲜橘子了，所以答案就是 0。
```

提示:

- $m = \text{grid.length}$
- $n = \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ 仅为 0、1 或 2

解决方法

1. 问题理解

目标： 计算所有新鲜橘子 (1) 被腐烂橘子 (2) 传染所需的最短时间（分钟数）。

规则：

- 每分钟，腐烂橘子会传染其上、下、左、右四个方向相邻的新鲜橘子。
- 空单元格 (0) 不参与。

输出：

- 最短分钟数。
- 如果有新鲜橘子永远不会腐烂，返回 -1。
- 如果一开始就没有新鲜橘子，返回 0。

2. 思路分析

这是一个典型的**多源广度优先搜索 (Multi-source BFS)** 问题。可以想象成腐烂过程是一层一层向外扩散的，BFS 正好能模拟这个过程并计算最短时间（层数）。

1. 初始化：

- 创建一个队列 `queue` 用于 BFS。
- 统计初始时新鲜橘子的数量 `fresh_count`。
- 遍历网格 `grid`：
 - 将所有初始腐烂橘子 (2) 的坐标 (r, c) 加入队列 `queue`。
 - 统计 `fresh_count`。
- 初始化时间计数器 `minutes = 0`。
- 获取网格尺寸 `rows, cols`。

2. 处理特殊情况：

- 如果 `fresh_count == 0`，说明没有新鲜橘子，直接返回 0。

3. 多源 BFS 过程：

- 当队列 `queue` 不为空时，进行循环（每一轮循环代表一分钟）：
 - 获取当前队列的大小 `level_size`，这代表当前这一分钟需要处理的（刚刚或之前腐烂的）橘子数量。
 - **处理当前层级的所有腐烂橘子：** 循环 `level_size` 次。
 - 将队首的腐烂橘子坐标 (r, c) 出队。
 - **检查并传染四个相邻单元格 $(next_r, next_c)$ ：**
 - 对于每个相邻单元格，进行边界检查 ($0 \leq next_r < rows$ and $0 \leq next_c < cols$)。
 - 检查该单元格是否是新鲜橘子 (`grid[next_r][next_c] == 1`)。
 - 如果是新鲜橘子：
 - 将其变为腐烂橘子：`grid[next_r][next_c] = 2`。
 - 将新鲜橘子计数减一：`fresh_count -= 1`。
 - 将被传染的橘子坐标 $(next_r, next_c)$ 入队，供下一分钟处理。
 - **时间增加：** 如果在这一分钟内队列处理了至少一个橘子（即 `level_size > 0`），则时间 `minutes += 1`。**注意：** 只有当队列非空时才增加时间，并且是在处理完一层之后增加。

4. 结束条件与结果判断：

- BFS 循环结束后（队列为空）：
 - 检查 `fresh_count` 是否为 0。

- 如果 `fresh_count == 0`，说明所有新鲜橘子都被腐烂了，返回 `minutes`。
- 如果 `fresh_count > 0`，说明有新鲜橘子无法被传染到，返回 `-1`。

细节注意：

- 为什么是多源 BFS？因为初始可能有多个腐烂橘子，它们同时开始传染。将所有初始腐烂橘子都放入队列即可实现多源同时开始。
- 如何计算时间？BFS 的层数就代表了时间。每一轮外层 `while` 循环处理完一层（即一分钟内发生的所有腐烂），然后时间加 1。
- 为什么在 `while` 循环内部记录 `level_size`？这是为了确保我们只处理当前这一分钟应该被处理的橘子，并将下一分钟才会被传染的橘子留在队列中，从而正确地按层（按分钟）推进。
- 时间 `minutes` 的增加应该在外层 `while` 循环的末尾，并且只有当 `level_size > 0` 时才增加（或者说，只要队列非空就处理一层，处理完一层就加一分钟）。但需要注意初始状态，如果一开始队列就非空，处理完第一层后 `minutes` 应该是 1。一个简单的处理方法是，如果最终 `fresh_count == 0`，返回 `minutes`；否则返回 `-1`。如果初始 `fresh_count == 0`，直接返回 0。

修正时间计算逻辑：

更清晰的时间计算方式是，只有当队列非空（意味着还有橘子在腐烂或传播）且在当前轮次中确实处理了橘子（`level_size > 0`）时，才将 `minutes` 加 1。

```

1  # ... BFS loop ...
2  while queue:
3      level_size = len(queue)
4      if level_size == 0: # 如果队列空了，提前结束
5          break
6
7      # 标记本轮是否有橘子被腐蚀（虽然 level_size > 0 已经隐含了）
8      processed_this_minute = False
9      for _ in range(level_size):
10         processed_this_minute = True # 只要循环执行了，就说明处理了
11         r, c = queue.popleft()
12         # ... 传染邻居 ...
13         # if grid[next_r][next_c] == 1:
14         #     ...
15         #     fresh_count -= 1
16         #     queue.append((next_r, next_c))
17
18     # 处理完一整层（一分钟）后，如果这一层处理过橘子，时间才增加
19     if processed_this_minute: # 或者直接 if level_size > 0:
20         minutes += 1

```

但是，如果最后一轮刚好把所有橘子腐烂完，`minutes` 会多加 1。

再修正时间计算逻辑：

在循环开始前 `minutes = 0`。在 `while queue:` 循环内部，先处理完一层，然后判断队列是否还非空。如果非空，说明下一分钟还有橘子要处理，此时 `minutes += 1`。

```

1  minutes = 0
2  while queue:
3      level_size = len(queue)
4      for _ in range(level_size):

```

```

5         r, c = queue.popleft()
6         # ... 传染邻居 ...
7         # if grid[next_r][next_c] == 1:
8         #     ...
9         #     fresh_count -= 1
10        #     queue.append((next_r, next_c))
11
12        # 处理完一层后，如果队列里还有东西（意味着下一分钟还要继续），时间才加1
13        if queue:
14            minutes += 1

```

这种方式下，如果最后一轮处理完，队列变空，minutes 就不会再增加了，得到的就是正确的时间。

最终确认时间计算逻辑：

最简单且不易出错的方法：

1. 初始化 minutes = 0。
2. 在 while queue: 循环外判断初始 fresh_count。
3. 在 while queue: 循环内，处理一层。
4. 在循环的末尾，只要这一轮处理了橘子 (level_size > 0)，就 minutes += 1。
5. 循环结束后，如果 fresh_count == 0，返回 minutes；否则返回 -1。

这种方法的问题在于，如果初始就有腐烂橘子，但没有新鲜橘子可传染，minutes 会变成 1，而期望是 0。

最佳时间计算逻辑：

1. 初始化 minutes = -1 (或者 0，然后在第一次有效传播后+1)。
2. 将所有初始腐烂橘子入队。
3. while queue:
4. minutes += 1 (每一轮开始时，时间加 1)
5. 处理当前层 level_size 个橘子，传染邻居并入队。
6. 循环结束后，判断 fresh_count。如果为 0，返回 max(0, minutes) (处理初始无新鲜橘子的情况)；否则返回 -1。

```

1  # 最佳逻辑实现
2  queue = collections.deque()
3  fresh_count = 0
4  rows, cols = len(grid), len(grid[0])
5
6  for r in range(rows):
7      for c in range(cols):
8          if grid[r][c] == 1:
9              fresh_count += 1
10         elif grid[r][c] == 2:
11             queue.append((r, c))
12
13  if fresh_count == 0:
14      return 0
15
16  minutes = -1 # 从 -1 开始，这样第一轮处理后变成 0
17  directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

```

```

18
19 while queue:
20     minutes += 1
21     level_size = len(queue)
22     for _ in range(level_size):
23         r, c = queue.popleft()
24         for dr, dc in directions:
25             next_r, next_c = r + dr, c + dc
26             if 0 <= next_r < rows and 0 <= next_c < cols and grid[next_r][next_c] ==
1:
27                 grid[next_r][next_c] = 2
28                 fresh_count -= 1
29                 queue.append((next_r, next_c))
30
31 return minutes if fresh_count == 0 else -1

```

这个逻辑看起来最合理。

3. 代码实现 (Python)

```

1 from typing import List
2 import collections
3
4 class Solution:
5     def orangesRotting(self, grid: List[List[str]]) -> int:
6         rows, cols = len(grid), len(grid[0])
7         queue = collections.deque()
8         fresh_count = 0
9
10        # 初始化队列和新鲜橘子计数
11        for r in range(rows):
12            for c in range(cols):
13                if grid[r][c] == 1:
14                    fresh_count += 1
15                elif grid[r][c] == 2:
16                    queue.append((r, c)) # 将初始腐烂橘子加入队列
17
18        # 如果一开始就没有新鲜橘子
19        if fresh_count == 0:
20            return 0
21
22        minutes = -1 # 时间从 -1 开始，这样第一轮传播后变为 0
23        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)] # 四个方向
24
25        # BFS 过程
26        while queue:
27            minutes += 1 # 进入新的一分钟
28            level_size = len(queue) # 当前这一分钟需要处理的腐烂源数量
29

```

```

30         for _ in range(level_size):
31             r, c = queue.popleft()
32
33             # 尝试感染四个方向的邻居
34             for dr, dc in directions:
35                 next_r, next_c = r + dr, c + dc
36
37             # 检查边界和是否为新鲜橘子
38             if 0 <= next_r < rows and 0 <= next_c < cols and grid[next_r]
[next_c] == 1:
39                 grid[next_r][next_c] = 2 # 变为腐烂
40                 fresh_count -= 1          # 新鲜橘子数减一
41                 queue.append((next_r, next_c)) # 加入队列，下一分钟会处理它
42
43             # 循环结束后，检查是否所有新鲜橘子都腐烂了
44             return minutes if fresh_count == 0 else -1
45

```

4. 复杂度分析

- **时间复杂度：** $O(M * N)$ ，其中 M 是行数， N 是列数。每个单元格最多被访问和处理常数次（入队出队一次）。
- **空间复杂度：** $O(M * N)$ ，在最坏情况下，所有橘子都可能加入队列中。

5. 如何在面试中讲解

1. 问题本质：

- "这个问题是求解一个多源最短路径问题。可以看作是腐烂状态从初始的腐烂橘子（多个源点）开始，一层层向外扩散到所有新鲜橘子的过程。"
- "我们需要找到所有新鲜橘子都被感染的最短时间，这自然联想到广度优先搜索 (BFS)。"

2. BFS 思路：

- "使用 BFS 来模拟腐烂过程。BFS 的层数就对应着时间（分钟数）。"
- "首先，需要找到所有初始的腐烂橘子，并将它们作为 BFS 的第一层（时间 0）加入队列。"
- "同时，统计出初始状态下新鲜橘子的总数 `fresh_count`。"

3. 多源 BFS 实现细节：

- "进行标准的 BFS 循环。每一轮外层循环代表一分钟。"
- "在每轮循环开始时，记录当前队列的大小 `level_size`，这代表当前这一分钟内作为传染源的橘子。"
- "然后，循环 `level_size` 次，处理这一层的所有腐烂橘子：将它们出队，并检查它们的四个邻居。"
- "如果邻居是新鲜橘子，就将其状态改为腐烂，`fresh_count` 减 1，并将该邻居入队，以便在下一轮（下一分钟）处理。"
- "用一个变量 `minutes` 记录时间。可以在每轮 BFS 外层循环开始时加 1（初始值为 -1），或者在处理完一层后加 1（初始值为 0，但需额外处理）。推荐从 -1 开始，每轮加 1。"

4. 结束条件与结果：

- "当 BFS 队列为空时，表示腐烂过程无法继续传播。"
- "此时，检查 `fresh_count`。如果为 0，说明所有橘子都腐烂了，返回 `minutes`。"
- "如果 `fresh_count` 大于 0，说明有些新鲜橘子永远无法被感染，返回 -1。"
- "需要处理一个边界情况：如果初始时 `fresh_count` 就为 0，直接返回 0。"

5. 复杂度分析：

- "时间复杂度 $O(M * N)$ ，每个格子最多入队出队一次。"
- "空间复杂度 $O(M * N)$ ，队列最坏情况下可能存储所有格子。"